quectel-Voice Call

版本：1.0

日期：2026-02-26

状态：受控/临时文件

文件保密级别：（请勾选 ■ ）		
绝密 □	保密 □	公开 ■

文件管控表

文件变更记录			
修订日期	最新版本	修订内容	编制
2026-03-06	1	首次发行	Wells Li

上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼　　邮编：200233
电话：+86 21 5108 6236　　邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：http://www.quectel.com/cn/support/sales.htm。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
http://www.quectel.com/cn/support/technical.htm 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬软件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2024，保留一切权利。
Copyright © Quectel Wireless Solutions Co., Ltd. 2024.

目录

1 模块概述 5

2 通话管理 5

2.1. Voice 5

2.1.1. 接口概述 5

2.1.2. 函数原型 5

2.1.3. 参数描述 5

2.1.4. 返回值描述 5

2.1.5. 示例 5

2.2. init 6

2.2.1. 接口概述 6

2.2.2. 函数原型 6

2.2.3. 参数描述 6

2.2.4. 返回值描述 6

2.2.5. 回调函数参数说明 6

2.2.6. 示例 6

2.3. deinit 7

2.3.1. 接口概述 7

2.3.2. 函数原型 7

2.3.3. 参数描述 7

2.3.4. 返回值描述 8

2.3.5. 示例 8

3 通话操作 8

3.1. dial 8

3.1.1. 接口概述 8

3.1.2. 函数原型 8

3.1.3. 参数描述 8

3.1.4. 返回值描述 8

3.1.5. 示例 8

3.2. answer 9

3.2.1. 接口概述 9

3.2.2. 函数原型 9

3.2.3. 参数描述 9

3.2.4. 返回值描述 9

3.2.5. 示例 9

3.3. hangup 10

3.3.1. 接口概述 10

3.3.2. 函数原型 10

3.3.3. 参数描述 10

3.3.4. 返回值描述 10

3.3.5. 示例 10

3.4. hold_and_answer 11

3.4.1. 接口概述 11
3.4.2. 函数原型 11
3.4.3. 参数描述 11
3.4.4. 返回值描述 11
3.4.5. 示例 11
3.5. release_and_answer 12
3.5.1. 接口概述 12
3.5.2. 函数原型 12
3.5.3. 参数描述 12
3.5.4. 返回值描述 12
3.5.5. 示例 12
4 常量定义 13

1 模块概述

Voice 模块提供了电话语音通话管理功能，包括拨打电话、接听电话、挂断电话等基本通话操作，以及通话状态回调通知功能，用于实现语音通话应用。

2 通话管理

2.1. Voice

2.1.1. 接口概述

创建一个新的 Voice 管理器对象。

2.1.2. 函数原型

voice = quectel.Voice()

2.1.3. 参数描述

无。

2.1.4. 返回值描述

返回一个新的 Voice 对象实例。

2.1.5. 示例

```
import quectel
voice = quectel.Voice()
```

2.2. init

2.2.1. 接口概述

初始化 Voice 管理器并注册回调函数。

2.2.2. 函数原型

result = voice.init(callback)

2.2.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
callback	是	function	-	通话状态回调函数，接收 4 个参数：id, dir, state, number

2.2.4. 返回值描述

初始化成功返回 True。

2.2.5. 回调函数参数说明

回调函数原型：callback(id, dir, state, number)

参数	类型	说明
id	int	通话 ID，唯一标识一个通话
dir	int	通话方向：0=主叫(MO)，1=被叫(MT)
state	int	通话状态（参考状态常量）
number	str	对方电话号码

2.2.6. 示例

```
from quectel import Voice
import time


def voice_cb(id, direction, state, number):
    if state == Voice.STATE_IDLE:
        state_str = "IDLE"
    elif state == Voice.STATE_HOLD:
```

```
            state_str = "HOLD"
        elif state == Voice.STATE_ORIGINAL:
            state_str = "ORIGINAL"
        elif state == Voice.STATE_CONNECT:
            state_str = "CONNECT"
        elif state == Voice.STATE_INCOMING:
            state_str = "INCOMING"
        elif state == Voice.STATE_WAITING:
            state_str = "WAITING"
        elif state == Voice.STATE_END:
            state_str = "END"
        elif state == Voice.STATE_ALERTING:
            state_str = "ALERTING"
        else:
            state_str = "UNKNOWN"

        # 呼叫方向
        if direction == Voice.DIR_MO:
            dir_str = "MO"
        elif direction == Voice.DIR_MT:
            dir_str = "MT"
        else:
            dir_str = "UNKNOWN"

    v = Voice()
    v.init(voice_cb)
    while True:
        time.sleep(1)
```

2.3. deinit

2.3.1. 接口概述

反初始化 Voice 管理器，释放资源。

2.3.2. 函数原型

```
voice.deinit()
```

2.3.3. 参数描述

无。

2.3.4. 返回值描述

无。

2.3.5. 示例

```
voice.deinit()
```

3 通话操作

3.1. dial

3.1.1. 接口概述

拨打电话。

3.1.2. 函数原型

```
result = voice.dial(phone_number)
```

3.1.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
phone_number	是	str	-	目标电话号码

3.1.4. 返回值描述

成功返回 True,否则返回 False。

3.1.5. 示例

```
import quectel
import utime

def call_callback(id, dir, state, number):
    print(f"状态: id={id}, dir={dir}, state={state}, number={number}")

voice = quectel.Voice()
voice.init(call_callback)

# 拨打电话
```

```
if voice.dial("10086"):
    print("拨号成功")

    # 等待通话结束
    utime.sleep(30)

else:
    print("拨号失败")

# 挂断电话
voice.hangup()
utime.sleep(3)
voice.deinit()
```

3.2. answer

3.2.1. 接口概述

接听来电。

3.2.2. 函数原型

```
result = voice.answer()
```

3.2.3. 参数描述

无。

3.2.4. 返回值描述

成功返回 True，失败返回 False。

3.2.5. 示例

```
import quectel
import time

def auto_answer(id, dir, state, number):
    if state == quectel.Voice.STATE_INCOMING:
        print(f"来电: {number}")
        voice.answer()  # 自动接听

voice = quectel.Voice()
voice.init(auto_answer)

print("等待来电...")
```

```
try:
    # 保持程序运行
    while True:
        time.sleep(1)

except KeyboardInterrupt:
    voice.deinit()
```

3.3. hangup

3.3.1. 接口概述

挂断当前通话。

3.3.2. 函数原型

voice.hangup()

3.3.3. 参数描述

无。

3.3.4. 返回值描述

无返回值。

3.3.5. 示例

```
import quectel
import utime

def call_callback(id, dir, state, number):
    print(f"状态: id={id}, dir={dir}, state={state}, number={number}")

voice = quectel.Voice()
voice.init(call_callback)

# 拨打电话
if voice.dial("10086"):
    print("拨号成功")

    # 等待通话结束
    utime.sleep(30)

else:
```

```
        print("拨号失败")

    # 挂断电话
    voice.hangup()
    utime.sleep(3)
    voice.deinit()
```

3.4. hold_and_answer

3.4.1. 接口概述

保持当前通话并接听等待中的来电。

3.4.2. 函数原型

```
result = voice.hold_and_answer()
```

3.4.3. 参数描述

无。

3.4.4. 返回值描述

成功返回 True，失败返回 False。

3.4.5. 示例

```
import quectel
import time

def auto_answer(id, dir, state, number):
    if state == quectel.Voice.STATE_WAITING:
        print(f"来电: {number}")
        voice.hold_and_answer()   # 自动接听

voice = quectel.Voice()
voice.init(auto_answer)
voice.dial('10086')
print("拨打电话并等待来电...")

try:
    # 保持程序运行
    while True:
        time.sleep(1)
```

```
except KeyboardInterrupt:
    voice.deinit()
```

3.5. release_and_answer

3.5.1. 接口概述

挂断当前通话并接听等待中的来电。

3.5.2. 函数原型

```
result = voice.release_and_answer()
```

3.5.3. 参数描述

无。

3.5.4. 返回值描述

成功返回 True，失败返回 False。

3.5.5. 示例

```
import quectel
import time

def auto_answer(id, dir, state, number):
    if state == quectel.Voice.STATE_WAITING:
        print(f"来电: {number}")
        voice.release_and_answer()   # 自动接听

voice = quectel.Voice()
voice.init(auto_answer)
voice.dial('10086')
print("拨打电话并等待来电...")

try:
    # 保持程序运行
    while True:
        time.sleep(1)

except KeyboardInterrupt:
    voice.deinit()
```

4 常量定义

常量名	值	说明
Voice.STATE_IDLE	0	空闲状态，无活动通话
Voice.STATE_HOLD	1	通话保持中
Voice.STATE_ORIGINAL	2	发起通话中
Voice.STATE_CONNECT	3	通话已连接
Voice.STATE_INCOMING	4	来电接入
Voice.STATE_WAITING	5	通话等待中
Voice.STATE_END	6	通话已结束
Voice.STATE_ALERTING	7	振铃中